

STANDING ADVISORY

 **Proof of concept, under active development.** An experiment, not production or regulatory tooling. It is **pre-1.0** and changes often; behaviour and interfaces may break between updates, and it can get things wrong. **Review everything it produces; never rely on it as a control or as regulatory advice.**

CASE FILE · QUICK-START REFERENCE

QUICK-START · NEW USERS

► compliance-surveillance-team · v0.33.6

The Compliance Surveillance Engineering Team

A virtual delivery team for transaction-monitoring, trade-surveillance and comms-surveillance engineering. Morgan (the PM) coordinates 16 named specialists to build, test, review and hand over solutions, keeping the evidence for each step as it goes.

RUNTIME GUARANTEES

Always on · agents are blocked from reading data/raw/

Always on · code under review is not run without your consent

Always on · all data is treated as PII/MNPI unless masked or synthetic

These guards run before every tool call, engaged or dormant. They are a real control against mistakes and prompt injection, not a sandbox against a determined attacker: see the safety model for what each one does and does not guarantee.

How this actually works

The whole mental model, no framework jargon required. *Dotted terms unfold: tap or hover for a plain-English definition.*

§1

An AI framework: one orchestrator, multiple subagents

Morgan orchestrates: planning, delegating. Mateo, Linh, Ravi and the rest are subagents, each with its own prompt, toolset and isolated context, seeing only its brief. That isolation is a feature: because specialists cannot see one another's reasoning, every review stays independent.

§2

Morgan coordinates the work rather than performing it herself

Morgan is the PM. She takes your request, decides which specialists the job actually needs, briefs them precisely, checks what comes back, and reports to you. You deal with Morgan directly; the rest of the team works behind the scenes.

§3

A loop is an iteration cycle, not a failure

§4

Talk to Morgan like you would a real PM

A failed check does not stop the work, it becomes another iteration until the work meets the standard. Same idea as a PR review: flagged, fixed, re-reviewed. The AI term is evaluator-optimizer: one pass produces the work, an independent pass checks it against a named standard, and it ships once that pass clears.

There is no command language to learn. Describe the outcome you want, in your own words. Morgan asks clarifying questions, flags anything unrealistic or risky, and checks in before anything material happens. Hand her context exactly as you would a colleague (a screenshot, a spec, an existing script) and she will work from it.

§5

 and  separate what was checked from what was reasoned

These are the tags you will see in every report, so they are worth knowing.  marks something verified directly: a test run, a query executed, a metric taken from the data.  marks reasoning that goes beyond the evidence, still useful, but flagged so it is not read as fact, which is the practical guard against hallucination.  marks a value read straight from source code, with nothing run.

§6

Every piece of work gets its own folder

Each engagement lives in `artifacts/<name>/`: its brief, specs, code, tests, and a living `START-HERE.md` status page showing where it stands, written to disk precisely so it survives compaction. Several can sit in the same project at once, each independent; one root index lists them all.

Getting started

The team is dormant by default. Nothing below happens until you ask for it. *Terminal panels are condensed from a real recorded run.*

1 /engage

Type this to start anything. It is the one command to remember.

```
CLAUDE
> /engage

Morgan (PM) hi, I'm Morgan, your PM on the Compliance
Surveillance Engineering team. We're on team v0.33.6.
Anytime you want the full roster, just type /meet-the-team.

NEW In the current release: a framework-audit remediation
pass. (Full detail: CHANGELOG.md)
```

2 Describe the work in plain English: a problem, a review, or something to build.

No need to name a specialist. Morgan classifies it and routes it.

```
CLAUDE
> A utility to de-duplicate our alert exports, with a
10-minute near-duplicate window. Samples are synthetic.

Classification: a clear build-from-spec, a small utility
with defined inputs and testable rules. Routing it as a build
engagement (spec, build, independent QA, code review,
compliance review), not a review.
```

- 3 **Answer the safety gate.** Two standing notices, then the questions batched into one screen. Nothing proceeds until you answer.

```
● ● ● CLAUDE

📖 🛡️ DATA SAFETY. 🧠 Everything you point me at goes to the
model provider. 🔴 Raw data is hard-blocked. 🟡 For anything
else you confirm it carries no PII/MNPI. 📺 I can't verify
that for you. 🟢 Unsure? Go synthetic.

[gate] Data safety: is the data involved safe to use?
[you ] Yes, synthetic, no prohibited PII

[gate] Execution: should the team run the test suite?
[you ] Yes, I'll grant consent

📖 Recorded. Intent alone unlocks nothing. When you're ready,
type this yourself: touch .claude/.exec-consent
No marker, no run, regardless of that answer.
```

- 4 **Watch the right-sizing.** Before any fan-out Morgan says how many specialists and why, so the cost is a stated decision rather than a surprise.

```
● ● ● CLAUDE

📖 Build's done, 12/12 of Kenji's own tests passed.

Right-sizing: engaging three specialists in parallel, each
an independent lens on the same code. Linh (QA) for boundary
evidence, Ravi (code review) for correctness and security,
Layla (compliance) for auditability. No further fan-out.
```

- 5 **A loop is normal.** Findings go back for a fix and get re-reviewed. The team says plainly when something is *not* finished.

```
● ● ● CLAUDE

📖 Critical: the drop logic only checks alert_id, never
whether the rows match. Two rows sharing an id but differing
in risk score: one is silently deleted. Worse, one of Kenji's
own tests locks that in as correct behaviour.

This is not closed, outstanding: two criticals to fix, plus
two business calls I will not guess at.

...fix pass, then the same three re-review...

📖 ✅ Closed, verdict: Ready. 0 open findings. 26 builder
tests + 30 independent QA tests passing. Everything in
artifacts/alert-dedup-utility/
```

The handful worth knowing

There are 24 commands. You do not need them: /engage reaches the rest for you. These are the ones worth recognising early. Type / in Claude Code to see them all, or read the full index in the [operating guide](#).

HAVE A LOOK FIRST

/demo

THE EVERYDAY WORK

/deep-review

watch the whole team work, on synthetic data, before trusting it with anything real

/meet-the-team

who the 16 specialists are and what each one owns

BEFORE ANY REAL DATA

/prepare-data ALPHA

Do not rely on this workflow. It is alpha: a lightweight, best-effort helper, **not** an anonymisation pipeline. Bring data your organisation has already masked with approved tooling, or fully synthetic data. Its synthetic generation is the part to trust; treat its masking as a rough first pass you still verify. Masked output remains personal data.

review existing code properly, before a PR or an audit

/new-scenario

a detection scenario from spec through build to sign-off

/analyse-data

a question you want evidenced, not yet an answer

/engage-light

a small fix that does not need the full ceremony

FINISHING

/handover

hand real code to a real dev or ops team, with the QA evidence

What every engagement leaves behind

The point of the ceremony. All of it on disk, in the engagement's own folder.

- ✓ **Engagement brief** the plan, decisions taken, open questions, who was engaged
- ✓ **The work itself** code, specs, config, each with its own tests
- ✓ **QA handover** independent test evidence, every cycle kept including failed ones
- ✓ **Review findings** what was raised, and how each one was dispositioned
- ✓ **Delivery report** the consolidated record, in .md and .html
- ✓ **Summary email** a short cover note stating the token and agent footprint
- ✓ **START-HERE index** a status page that is generated, never hand-written

What Morgan cannot do

Worth knowing, because these are enforced in code rather than promised in a prompt.

- ✗ **Grant permission to run your code.** The consent marker is yours to create; she is blocked from writing it, and says so rather than asking you to work around it.
- ✗ **Edit her own guardrails.** Writes to the safety hooks and the harness config are refused.
- ✗ **Sign off her own work.** Human sign-off is the one item on the Definition of Done that only you can complete.
- ✗ **Close an engagement with findings open.** The close runs the mechanical gate and refuses, rolling back, until it is clean.
- ✗ **Quietly leave something unfinished.** A turn that ends waiting on you is recorded as blocked, with the outstanding list named.

The read and execution guards are strong controls on the file tools and a real deterrent on the shell, but they are lexical there rather than a sandbox. The [safety model](#) states exactly what each one guarantees.

What people ask first

The questions that come up before anyone trusts a team of agents with real work.

Will it change my code without asking?

Builders write files; the reviewers and specialists who advise cannot edit anything, by tool grant rather than by request. Work lands in the engagement's own folder unless you ask for it in your source tree, nothing is committed or pushed for you, and each phase stops at a gate for your answer.

What does one engagement cost, and how long?

Across the 12 engagements in our own most recent test run: **\$0.53 to \$16.82** (median about \$4) and **3 to 42 minutes** (median 13). Those were headless runs that cannot reuse the specialists' prompt cache, so an interactive session should cost less. Bigger scope means more specialists, and more specialists means more tokens.

Where does my data go?

Anything you point the team at is sent to the model provider as context, the same as any other Claude session. The protected raw-data folder is blocked outright; everything else is your judgement call, which is why the sheet keeps saying to mask or synthesise first.

Why a team, rather than just asking Claude?

One assistant checking its own output is marking its own homework. Here the reviewer never saw the builder's reasoning, so the second opinion is genuinely independent. That separation is the entire point, and it is also why this costs more than a single prompt.

Will I get the same answer twice?

Not word for word. The mechanical checks are deterministic: same input, same result, every time. The judgement parts vary at the margin, so wording, ordering and genuinely borderline findings can shift between runs, while a clear-cut one is far more stable. Treat any single run as one careful pass rather than an exhaustive sweep: reports record the model and version, and absence of a finding is not evidence of absence. If something is load-bearing, verify it rather than relying on one pass.

Can I stop it half way?

Yes, interrupt whenever you like. Anything already written stays on disk and the engagement records itself as blocked rather than finished, so it never quietly looks complete. A later session reads that state and picks up where it stopped.

Best practices

- **Mention your materials:** a screenshot, a spec, an existing script. Morgan works from what you hand her, and guesses less when you hand her more.
- **Say what "done" means:** review only, or also fixed, or also handed over. Agreed before work starts beats renegotiated after.

- **Push back.** Morgan is meant to be a sceptic rather than a yes-man, and so are you. Multi-agent work costs roughly 15x the tokens of a single pass, which is why the team stays lean. If a finding looks thin, ask what the evidence actually was.
- **Check the tags.** 📊 was measured, 🧠 was reasoned. If something important is 🧠, that's your cue to verify it before relying on it.

- **Do not hand it unmasked data.** Bring data already masked by your organisation's approved tooling, or go fully synthetic. `/prepare-data` is alpha and should not be relied on for it. Pseudonymised is not anonymous.
- **It is a starting point.** A proof of concept whose output is for real engineers and reviewers to check, not a replacement for them.

github.com/danieledge/virtual-surv-IT

Deeper detail: [docs/OVERVIEW.md](#) (plain English) · [the operating guide](#) (standing rules and routing) · [the engagement-flow poster](#) (the same journey as a technical diagram, plus guards, repo checks and evals).